

Corrigé de l'Examen POO pour le réseau

du 25 Mai 2023

RSI M1

Question 1 : 1pt

Dans une communication basée sur les sockets, un client voudrait envoyer les informations concernant un Employé (nom, prénom, adresse) vers le serveur. Où est ce que la classe employé doit être créée ?

- a. Coté serveur
- b. Coté client
- c. Des deux cotés

Question 2 : 1pt

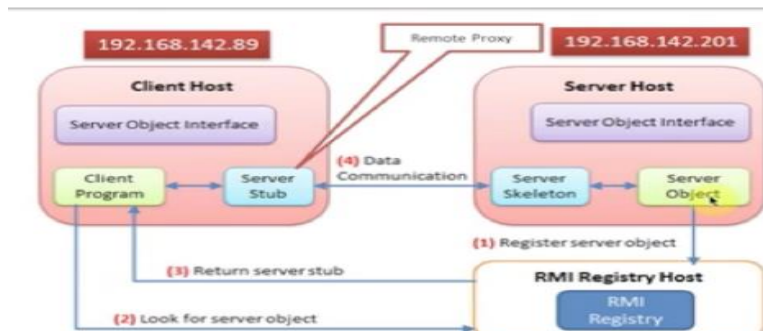
Quelle est l'opération indispensable pour les 2 communiquant, s'ils veulent s'envoyer des objets de types Employé ?

- a. Le downcasting, du type Object vers le type Employé
- b. La sérialisation du type Employé
- c. Le upcasting du type Employé vers le type Object
- d. Toutes les réponses a, b, c sont correctes

Question 3 : 2pt

On a étudié le DP Remote Proxy pour RMI. Effectivement, dans RMI, que représente la classe Proxy et la classe réelle, en prenant en compte le diagramme de classe du Proxy DP vu en cours ?

Reponse 3 : Dans RMI, la Classe Proxy(Remote) représente la classe server Stub chez le client, la classe réelle est la classe implémentant l'objet distant, chez le serveur.(voir image ci-dessous, tirée du cours RMI)





Question 4 : 2pt

On a appliqué le DP Decorator dans le TP CoffeeShop que vous avez validé avec RMI. Pourquoi à votre avis l'installation des classes -développées dans le cours Decorator- coté serveur et coté client est-elle une très mauvaise solution de programmation ? Quelle est la solution logique ?

Reponse 4

L'installation des classes -développées dans le cours Decorator- coté serveur et coté client enfreint le principe même du client/serveur, qui stipule que le Client demandant un service, ne peut pas le créer lui-même. C'est **le serveur qui doit se charger des détails d'implémentation**. Il suffit dans notre cas au Client **d'appeler une méthode**, que le serveur a implémenté, afin de visualiser le service. Cette méthode doit être disponible dans une interface commune aux Client et Serveur.

Question 5 3pt

Dans les 2 exercices sur le DP Factory (Thali et Pizza Franchise), l'invocation de la méthode qui crée la factory était différente, quelle est à votre avis la meilleure solution et pourquoi ?

Reponse 5

La solution de l'exercice Pizza store sépare la création de la factory du reste du processus de fabrication, par contre, dans l'exercice Thali, le processus est inclus dans la méthode Factory, du coup, la première solution est plus optimale car elle respecte le principe open-close.

Question 6 3pt

Le directeur d'une entreprise voudrait créer un mailing box, où il sera le seul à visualiser les mails de ses employés, chaque fois qu'il y a un nouveau message. Modifiez l'exercice 5 de la série 4 pour que cela soit possible

Corrigé Question 6 : le changement le plus important doit s'effectuer sur la classe implémentant l'interface Observer qui devrait être un Singleton, un seul observateur doit être créé : le directeur

```
+++++++observateurinterface+++++++
+++
public interface Observer {
    public void update();
    public void setSubject(Subject subject); }
+++++++ observateur concret
=+++++++
```

```
public class Director implements Observer {
    private static Director director;
    private String email;
```

```
private Subject mailingBox;
private Director(){}
public static Director getDirector(){
    if director==null director= new Director()
    return director;
}
public void setSubject(Subject mailingBox) {
    this.mailingBox = mailingBox;
    email= "No New email"; }

@Override
public void update() {
    System.out.println("State change reported
    by Subject.");
```



```
email= (String) mailingBox.getUpdate(); }
```

```
public String getEmail() {
```

```
return email; }
```

```
++++++= le sujet Observable : interface  
=++++++=
```

```
public interface Subject {
```

```
public void registerObserver(Observer  
observer);
```

```
public void notifyObserver();
```

```
public void unregisterObserver(Observer  
observer);
```

```
public Object getUpdate(); }
```

```
++++++implementation de l'objet  
observable++++++
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class MailingBox implements Subject {
```

```
List observersList;
```

```
private boolean stateChange;
```

```
public MailingBox () {
```

```
this.observersList = new ArrayList();
```

```
stateChange = false; }
```

```
public void registerObserver(Observer observer)  
{
```

```
observersList.add(observer); }
```

```
public void unregisterObserver(Observer  
observer) {
```

```
observersList.remove(observer); }
```

```
public void notifyObserver() {
```

```
if (stateChange) {
```

```
for (Observer observer : observersList) {
```

```
observer.update();
```

```
}}
```

```
}
```

```
public Object getUpdate() {
```

```
Object changedState = null;
```

```
if (stateChange) {
```

```
changedState = "Director Observing Mails"; }
```

```
return changedState; }
```

```
public void postNewEmail() {
```

```
stateChange = true;
```

```
notifyObserver(); }
```

```
++++++= Class Test =++++++
```

```
public class Test{
```

```
public static void main(String args[]) {
```

```
MailingBox mailingBox= new MailingBox();
```

```
director= Director.getDirector;
```

```
mailingBox.registerObserver(director);
```

```
director.setSubject(mailingBox);
```

```
System.out.println(director.getEmail());
```

```
mailingBox.postNewEmail();
```

```
System.out.println(director.getEmail()); }
```



Problème :: 8Pts : pour les 2 questions, dessiner le diagramme de classe et l'implémentation la plus explicite.

Q1 :: **DP :1Pt, Diag : 1pt, Impl : 2Pts**----- Q2 **DP :1Pt, Diag : 1pt, Impl : 2Pts**

On voudrait construire une application qui détecte 4 types d'attaques web : XSS, Sql Injection, Webshell et phishing. On va les appeler XssShield, SqlInjshield, WebshellShield et PhishingShield. On supposera que toutes ces attaques sont lancées à partir de scripts Php, qui doivent subir un ensemble d'opérations de prétraitement, ensuite de vectorisation et enfin d'entraînement sur un modèle approprié. Proposez un modèle (DP) dont le but sera de distinguer entre ces 4 attaques.

Cette application a réalisé des taux de détections assez remarquables, du coup, les développeurs ont voulu lui rajouter un module de traitement d'une nouvelle attaque, "Security Misconfiguration " quel DP préconisez-vous ? Mettre à jour la 1^{ère} question en concordance.

NB. De préférence, commencez d'abord par résoudre la 2eme question.

Solution du problème :

La première partie se résout avec le DP Chain of responsibility, l'application doit tester un après l'autre les modèles existant pour savoir le type d'attaque

La deuxième partie se résout avec le DP Factory, car il y a un processus de fabrication qui s'applique à tous les modèles de detection(pre-traitement, vectorisation, entraînement) et qui doit être séparé de l'instanciation du modèle lui-même.

***** deuxième partie *****Factory, abstract class*****

```
Public abstract class AbstractShieldFactory{
```

```
    abstract Model createModel(String item) ;
```

```
    public Model modelize(String type){
```

```
        Model model=createModel(type);
```

```
        pre-treat();
```

```
        vectorize();
```

```
        train();
```

```
        test();
```

```
        Return model;
```

```
    } }
```



*****Factory, concrete Factory class*****

Public class **ShieldFactory** extends **AbstractShieldFactory**{

String filename;

Public ShieldFactory(String filename){

This. filename = filename ;}

Model createModel(String item) {

If (item.equals("Webshell")) {return new WebshellModel(String fileName);}

else if (item.equals("Phishing")) {return new PhishingModel (String fileName);}

else if (item.equals("XSS")) {return new XSSModel (String fileName);}

else if (item.equals("SqlInject")) {return new SqlInjectModel (String fileName);}

else if (item.equals("MisconfigurationSec")) {return new MisconfigurationSecModel (String fileName);}

} }

*****Product, Abstract class*****

Public abstract class Model{

Double tauxDetect=0 ;

void pre-treat(){System.out.println("Pre-treating the php file"); }

void vectorize (){System.out.println("Vectorizing the php file"); }

void train (){System.out.println("Training the php file"); }

void test (){System.out.println("Testing the php file and calculating detection score");}

double getDetectScore(){return tauxDetect;}

}

}

*****Product, concrete classes *****

Public class WebshellModel extends **Model**{

Public WebshellModel(String fileName){

System.out.println("Applying Webshell model on the"+fileName+"Php file");

}}

Public class PhishingModel extends **Model**{

Public PhishingModel (String fileName){



```
System.out.println("Applying Phishing model on the"+fileName+"Php file");
```

```
}}
```

```
Public class XSSModel extends Model{
```

```
public XSSModel (String fileName){
```

```
System.out.println("Applying XSS Model on the"+fileName+"Php file");
```

```
}}
```

```
Public class SqlInjectModel extends Model{
```

```
public SqlInjectModel (String fileName){
```

```
System.out.println("Applying SqlInject Model on the"+fileName+"Php file");
```

```
}}
```

```
Public class MisconfigurationSecModel extends Model{
```

```
public MisconfigurationSecModel (String fileName){
```

```
System.out.println("Applying MisconfigurationSec Model on the"+fileName+"Php file");
```

```
}}
```

```
***** première partie *****CoR, abstract class*****
```

```
public abstract class AttackHandler {
```

```
private AttackHandler successor;
```

```
public void setSuccessor( AttackHandler successor ) {
```

```
this.successor = successor; }
```

```
public AttackHandler getSuccessor() {
```

```
return successor; }
```

```
public void detectAttack() { successor. detectAttack (); } }
```

```
*****CoRConcrete classes*****
```

```
public class WebshellShield extends AttackHandler {
```

```
public void detectAttack () {
```



```
AbstractShieldFactory shieldFactory= new ShieldFactory("Test.php");
```

```
Model model=shieldFactory.modelize("Webshell");
```

```
tauxDetect= model.getDetectScore();
```

```
if( tauxDetect <0.5) {getSuccessor().detectAttack () else { System.out.println("This attack is Webshell") }}
```

```
public class PhishingShield extends AttackHandler {
```

```
public void detectAttack () {
```

```
AbstractShieldFactory shieldFactory= new ShieldFactory("Test.php");
```

```
Model model=shieldFactory.modelize("Phishing");
```

```
tauxDetect= model.getDetectScore();
```

```
if( tauxDetect <0.5) {getSuccessor().detectAttack () else { System.out.println("This attack is Phishing") }}
```

```
public class XSSShield extends AttackHandler {
```

```
public void detectAttack () {
```

```
AbstractShieldFactory shieldFactory= new ShieldFactory("Test.php");
```

```
Model model=shieldFactory.modelize("XSS");
```

```
tauxDetect= model.getDetectScore();
```

```
if( tauxDetect <0.5) {getSuccessor().detectAttack () else { System.out.println("This attack is XSS") }}
```

```
public class SqlInjShield extends AttackHandler {
```

```
public void detectAttack () {
```

```
AbstractShieldFactory shieldFactory= new ShieldFactory("Test.php");
```

```
Model model=shieldFactory.modelize("SqlInject");
```

```
tauxDetect= model.getDetectScore();
```

```
if( tauxDetect <0.5) {getSuccessor().detectAttack () else { System.out.println("This attack is SqlInjection") }}
```



```
public class MisconfigShield extends AttackHandler {
```

```
public void detectAttack () {
```

```
AbstractShieldFactory shieldFactory= new ShieldFactory("Test.php");
```

```
Model model=shieldFactory.modelize("MisconfigurationSec");
```

```
tauxDetect= model.getDetectScore();
```

```
if( tauxDetect <0.5) { System.out.println("This attack cannot be detected or is no-attack")} else { System.out.println("This  
attack is Misconfiguration") } }}
```

```
public class TestShield {
```

```
public static void main( String arg[] ) {
```

```
AttackHandler webshell = new WebshellShield ();
```

```
AttackHandler phishing = new PhishingShield ();
```

```
AttackHandler xss = new XSSShield ();
```

```
AttackHandler sqlInj = new SqlInjShield ();
```

```
AttackHandler misconfg = new MisconfigShield ();
```

```
webshell.setSuccessor( phishing);
```

```
phishing.setSuccessor( xss);
```

```
xss.setSuccessor( sqlInj);
```

```
sqlInj setSuccessor( misconfg);
```

```
webshell. detectAttack (); }
```

```
}
```

NB. Toutes les classes de ce problème doivent être dans le même projet/package

